Capítulo 10: Los Entresijos Internos de Git

Actividad 2 - Basado en Pro Git (Chacon & Straub, 2014) - Juan Manuel Gomez Perez - 43972

Introducción

Este capítulo explica cómo funciona Git por dentro: de qué manera almacena los datos, qué son los objetos y las referencias, cómo se empaquetan y cómo se transfieren. Entender esto ayuda a usar mejor Git y a recuperar información en situaciones difíciles.

10.1 Fontanería y porcelana

Git tiene dos tipos de comandos. Los de porcelana son los amigables de uso diario (add, commit, push). Los de fontanería (plumbing) son comandos de bajo nivel que realizan las tareas internas y sirven de base para los primeros.

10.2 Objetos de Git

Git es un sistema de archivos direccionable por contenido: guarda los datos en la carpeta .git/objects y los identifica con un hash SHA-1 calculado a partir del contenido. Hay cuatro tipos de objetos:

- blob: el contenido de un archivo.
- tree: un directorio; lista blobs y otros árboles con sus nombres.
- commit: apunta a un árbol e incluye autor, fecha, mensaje y padre(s).
- tag: una etiqueta anotada que apunta a otro objeto.

```
git hash-object -w archivo.txt
git cat-file -p <hash>
git cat-file -t <hash>
git update-index --add --cacheinfo 100644 <hash> archivo.txt
git write-tree
git commit-tree <hash-arbol> -m "Mensaje"
```

10.3 Referencias de Git

Las referencias son archivos que contienen un hash y le dan un nombre legible a una confirmación. Viven en .git/refs: heads (ramas), tags (etiquetas) y remotes (ramas remotas). HEAD es una referencia simbólica que apunta a la rama actual.

```
git update-ref refs/heads/prueba <hash>
git symbolic-ref HEAD
```

10.4 Archivos empaquetados (packfiles)

Al principio Git guarda cada objeto suelto. Para ahorrar espacio, los agrupa en archivos pack y guarda los objetos similares como diferencias (deltas). Esto ocurre al ejecutar git gc o al enviar datos a un servidor.

```
git gc
```

```
git count-objects -v
git verify-pack -v .git/objects/pack/<archivo>.idx
```

10.5 Las especificaciones de referencia (refspec)

Un refspec define cómo se corresponden las referencias remotas con las locales. Por ejemplo, +refs/heads/*:refs/remotes/origin/* indica que las ramas del remoto se guardan como origin/<rama>. Permite configurar qué se descarga o se envía.

10.6 Protocolos de transferencia

Git usa un protocolo "tonto" (solo descarga archivos por HTTP) y uno "inteligente", donde cliente y servidor negocian qué datos faltan. Para subir se usan procesos como send-pack y receive-pack; para bajar, fetch-pack y upload-pack.

10.7 Mantenimiento y recuperación de datos

```
git gc --auto
git fsck --full
git prune
git reflog
```

git fsck comprueba la integridad de la base de datos y encuentra objetos inaccesibles. Con ayuda del reflog suele ser posible recuperar confirmaciones que parecían perdidas. git gc elimina objetos que ya no se necesitan tras cierto tiempo.

10.8 Variables de entorno

Git se puede controlar con variables como GIT_DIR (ubicación del directorio .git), GIT_AUTHOR_NAME, GIT_AUTHOR_EMAIL, GIT_COMMITTER_NAME, GIT_EDITOR o GIT_TRACE (para depurar). Son útiles en scripts y para modificar el comportamiento de Git sin cambiar la configuración.

Fuente: Chacon, S., & Straub, B. (2014). Pro Git (2.ª ed.). Apress.
https://git-scm.com/book/es/v2